



CESDE

Santiago Yosa

Ingeniero Sistemas de Información



Unidad II

JPA+HIBERNATE CONEXIÓN CON BD

- Configuración conexión con BD
- Mapeo de entidades
- Relaciones entre entidades
- Operaciones de persistencia CRUD

SISTEMA CONTROL DE VERSIONES

- Utilizar git/GitHub para versionar código realizado

Fundamentos de POO

Puente Cognitivo

Del bloque anterior, debemos dominar qué es una **Clase** (el plano/molde) y qué es un **Objeto** (la instancia en memoria con estado/atributos concretos). También deben recordar el uso fundamental de Git (add, commit).

Reto rápido

Imaginen que tienen un objeto Factura en Java, el cual tiene una lista de objetos ItemFactura. Ahora imaginen una base de datos relacional (tablas de Excel). Si intento guardar esa factura entera en una sola fila de la tabla.

¿Qué problema fundamental me encuentro al intentar meter la 'lista de ítems' en una sola celda?

No se puede. Las tablas son planas (filas y columnas) y los objetos Java son tridimensionales (tienen listas, herencia y otros objetos dentro).

Situación - El Código Espagueti de las Conexiones Manuales:

Imagina que trabajamos para una clínica y nos piden guardar un Paciente en la base de datos. En el pasado (usando JDBC estándar), un desarrollador tenía que:

1. Escribir la conexión manual a la base de datos (usuario, contraseña, URL).
2. Escribir un String gigante con código SQL puro: "INSERT INTO pacientes (nombre, edad) VALUES ('" + paciente.getNombre() + "', " + paciente.getEdad() + ")".
3. Ejecutar el código y gestionar manualmente si la base de datos se caía. *¿El problema?* Si cambiabas el nombre de la variable edad a fechaNacimiento en Java, el SQL no te avisaba del error hasta que el programa explotaba en la cara del cliente. Además, mezclar Java con textos en SQL hace que el código sea inmantenible.

Usar un framework que tome nuestro objeto Java y, automáticamente, lo traduzca y lo guarde en la tabla SQL sin que nosotros escribamos una sola línea de SQL. A esto lo llamamos ORM (Spring Data JPA / Hibernate). Para usarlo, primero necesitamos preparar el ecosistema (Spring Boot y Maven).

El Paradigma ORM

Mapeo Objeto-Relacional (ORM): Es una técnica de programación que permite convertir datos entre un sistema de tipos incompatibles (Java vs Base de Datos SQL).

El problema que resuelve se llama "**Desajuste de Impedancia**": Java es Orientado a Objetos (usa herencia, colecciones, referencias) y SQL es Relacional (usa llaves primarias, foráneas, tablas planas).

La equivalencia ORM:

- Una **Clase** en Java == Una **Tabla** en SQL.
- Un **Objeto** en Java == Una **Fila (Registro)** en SQL.
- Un **Atributo** en Java == Una **Columna** en SQL.

Gestión de Dependencias (Maven y pom.xml)

Históricamente, los programadores descargaban archivos .jar de internet y los pegaban en sus proyectos para poder usar librerías de terceros. **Maven** eliminó esto. Es un gestor de proyectos que centraliza las dependencias. Tú escribes qué librería necesitas en un archivo de configuración llamado pom.xml, y Maven viaja a internet, la descarga y la conecta a tu proyecto mágicamente.

- **groupId**: El nombre de la organización (ej. co.edu.uniminuto).
- **artifactId**: El nombre de la librería o del proyecto (ej. sistema-clinica).

El Ecosistema Spring Boot y Maven Wrapper (mvnw)

Spring Boot es un framework que "toma decisiones" por ti (Convención sobre Configuración) para que no tengas que configurar servidores desde cero. Todo proyecto de Spring se genera usando la web *Spring Initializr*.

El Wrapper (mvnw): En las empresas, instalar la herramienta "Maven" globalmente en cada computadora causa problemas de versiones. Spring te regala un archivo llamado mvnw (Maven Wrapper).

Este archivo es un "script" que **descarga un Maven temporal** en el fondo y compila tu proyecto. Así, los estudiantes pueden compilar código en los computadores del aula de clases sin requerir permisos de administrador para instalar programas.

¿Qué es una Dependencia?

Es cualquier biblioteca, framework o módulo de software externo que tu proyecto necesita para compilar, ser probado o funcionar.

¿Para qué sirve un Gestor de Dependencias?

Es una herramienta que **automatiza** el proceso de declarar, descargar, configurar y gestionar las dependencias de un proyecto.

Archivo de Configuración: pom.xml (Project Object Model). Es un archivo XML donde declaras todo sobre tu proyecto.

Las Coordenadas Maven: Cada biblioteca se identifica por tres piezas únicas:

- `<groupId>`: El "apellido" del proyecto, usualmente el nombre de la organización (ej. org.junit.jupiter).
- `<artifactId>`: El "nombre" del proyecto, el nombre de la biblioteca (ej. junit-jupiter-api).
- `<version>`: La versión específica que necesitas (ej. 5.9.1).

Estructura de Carpetas:

- **src/main/java**: Tu código fuente de la aplicación.
- **src/main/resources**: Archivos de configuración, imágenes, etc.
- **src/test/java**: Tu código de pruebas unitarias.
- **target**: Carpeta donde Maven genera los archivos compilados (.class) y el paquete final (.jar).

El Rol Mágico de Spring Boot

Sin Spring Boot, configurar JPA/Hibernate requiere un archivo complejo llamado persistence.xml. Spring Boot, fiel a su filosofía de "convención sobre configuración", simplifica esto drásticamente: si ve las dependencias de JPA en el pom.xml, auto-configura casi todo. Nuestra única tarea es proveer los detalles de la conexión en un archivo mucho más simple: application.properties.

```
spring.application.name=inventario
# Configuración de la base de datos SQL Server
spring.datasource.url=jdbc:sqlserver://SACHI:1433;databaseName=miinventario;trustServerCertificate=true; (Cadena conexión de tu servidor)
spring.datasource.username=tuusuario (tu usuario) SQL server
spring.datasource.password=tuclave (tu contraseña) SQL server
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver
# Propiedades de Hibernate para JPA
# 'update' crea o actualiza las tablas automáticamente al iniciar la app
spring.jpa.hibernate.ddl-auto=update
# Muestra las sentencias SQL en la consola
spring.jpa.show-sql=true
# Especifica el dialecto de Hibernate para SQL Server
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.SQLServerDialect
```

Control de Versiones y el Archivo .gitignore

Cuando compilamos el código (usando `.\mvnw clean compile`), Maven lee nuestro código fuente (`src/`) y genera el código binario (`.class`) dentro de una carpeta llamada **target/**.

- **Regla de Oro Corporativa:** JAMÁS se sube código compilado (la carpeta `target/`), ni configuraciones del IDE (como la carpeta `.idea` o `.vscode`) ni entornos virtuales al repositorio de Git.
- **La Solución:** Utilizar correctamente el archivo `.gitignore` indicando explícitamente estas carpetas para que Git sea "ciego" a ellas. Esto mantiene el repositorio limpio y evita conflictos.

```
### Maven (El código compilado) ###
```

```
target/
```

```
!.mvn/wrapper/maven-wrapper.jar
```

```
!**/src/main/**/target/
```

```
!**/src/test/**/target/
```

```
### VS Code (Configuraciones del editor) ###
```

```
.vscode/
```

```
*.code-workspace
```

```
.history/
```

```
git rm -r --cached target/
```


Preguntas

Si sé hacer consultas SQL complejas de forma excelente, ¿por qué me obligan a usar un ORM que parece esconder la base de datos?

El ORM no reemplaza el SQL complejo (puedes inyectar SQL nativo si lo necesitas). El ORM está diseñado para manejar el 80% del trabajo aburrido y repetitivo (los CRUDs). Aumenta dramáticamente la velocidad de desarrollo, previene ataques de inyección SQL y permite que, si mañana la empresa decide cambiar de MySQL a Oracle, no tengas que reescribir todo tu código (el ORM lo traduce automáticamente al nuevo dialecto).

¿Qué pasa si borro sin querer el archivo mvnw de mi proyecto?

Tu código fuente (el Java) estará a salvo, pero ya no podrás compilar ni correr el programa desde la terminal a menos que instales Maven de forma global en tu máquina. El Wrapper es la "grúa" que levanta el proyecto; si la borras, tendrás que descargar otro proyecto de Spring Initializr para recuperar ese archivo.

Ejemplos Demostrativos (Paso a Paso)

Creación estructural de un proyecto Spring.

Paso 1: Generación (Spring Initializr)

1. Entrar a start.spring.io
2. Seleccionar: **Maven, Java**, Spring Boot
3. Group: co.edu.cesde | Artifact: demo-persistencia
4. Dependencies: Spring Data JPA y H2 Database.
5. Clic en *Generate*. Extraer el .zip.

Paso 2: Anatomía del pom.xml

Práctica Acompañada - Learning by Doing

Vamos a preparar nuestro entorno para el trabajo de estas 4 sesiones.

1. Ingresa a start.spring.io.
2. Replica la configuración mostrada por el docente: Maven, Java, y agrega las dependencias Spring Data JPA y H2 Database.
3. Descarga el ZIP.
4. Descomprímelo en tu carpeta de trabajo.
5. Abre el proyecto en tu IDE

Práctica B: Afianzamiento (El Git Limpio)

Vamos a garantizar que el código se guarde correctamente sin basuras.

1. En la misma terminal de tu proyecto, inicializa git: `git init`.
2. Verifica el estado: `git status`.
3. Creemos nuestro `gitignore`
4. Modifica la clase principal `...Application.java` agregando dentro del `main`.
`System.out.println("Ecosistema Spring Boot Inicializado con Éxito");`
5. Agrega los cambios: `git add .`
6. Crea el commit base: `git commit -m "inicializacion de proyecto spring data"`
7. Ejecuta la aplicación

Cacería de Bugs (Mob Debugging):

Un estudiante hizo git status y la consola muestra que hay 500 archivos listos para hacer commit dentro de la carpeta target/classes/

Pregunta

¿Por qué la carpeta target/ se está colando en Git si dijimos que debía ignorarse?

Posiblemente borramos accidentalmente el archivo .gitignore o escribió mal la regla (ej. escribió Target/ con mayúscula inicial).

O no realizamos la creación correcta del archivo .gitignore

Se debe restaurar el archivo .gitignore y ejecutar git rm -r --cached target/ para limpiar la memoria de Git.

Ejercicio Independiente Aplicado

Eres el Tech Lead de un nuevo proyecto. Debes crear la plantilla base (Template) para el sistema de biblioteca de la Universidad.

Instrucciones de la Actividad:

1. Ve a Spring Initializr y crea un proyecto nuevo con Artifact: sistema-biblioteca.
2. Agrega las dependencias: Spring Data JPA y H2 Database.
3. Al descomprimirlo y abrirlo en tu IDE, ubica el archivo `application.properties` (dentro de `src/main/resources`).
4. Inicializa un repositorio Git local.
5. Haz un commit inicial.
6. Compila el proyecto
7. Crea un repositorio público en GitHub llamado `sistema-biblioteca-jpa` y sube tu código (`git push`).

Criterios de Logro Esperados (Evaluación):

1. **Control de Versiones estricto:** Al revisar el repositorio en GitHub del estudiante, **no debe existir** la carpeta `target/`.
2. **Estructura intacta:** El archivo `mvnw` y el `pom.xml` deben estar presentes en la raíz del repositorio de GitHub.
3. **Validación de Ecosistema:** El proyecto debe compilar limpiamente en la máquina del docente ejecutando `.\mvnw compile` al clonarlo.

CESDE | comfama